Chapter 8 | Exercise solutions with Python

From the Original Block to Modern Decoders

This notebook solves Exercises 6–11, including two optional coding exercises. It checks the effect of epsilon in RMSNorm, rebuilds a SwiGLU example from its weights, and measures how a causal decoder handles later tokens and padding. It derives the decoder's parameter count, simulates positions for keys entering a cache, and traces a gradient through two residual arrangements. Exercises 1–5 and 12 are in the written solutions.

To run the notebook, open it inside a copy of `transformers-book` and select a Python kernel with the packages in `requirements-ch05.txt`. Run the cells in order. The exercises run on CPU and include their saved outputs.

```
from pathlib import Path
import sys

import torch
from torch import nn
from torch.nn import functional as F

working_directory = Path.cwd()
for repo_root in (working_directory, *working_directory.parents):
    model_path = (
        repo_root / "examples" / "ch08_modern" / "model.py"
    )
    if model_path.is_file():
        sys.path.insert(0, str(repo_root))
        break
else:
    raise FileNotFoundError(
        "Open this notebook inside transformers-book."
    )

torch.set_num_threads(1)
```

6. Inspect a small denominator

Take a two-feature token $x = (3 \cdot 10^{-4}, 4 \cdot 10^{-4})$. RMSNorm divides both entries by a shared scale computed from this token alone. Let s be the average of the two squared entries. The denominator is $\sqrt{s + \epsilon}$, with $\epsilon = 10^{-6}$. The unit gains multiply each coordinate by one after the division.

If $y_i = x_i/\sqrt{s + \epsilon}$, then both y_i^2 contain the same factor $1/(s + \epsilon)$. Take that factor outside the average to obtain the output mean square.

$$\frac{y_1^2 + y_2^2}{2} = \frac{(x_1^2 + x_2^2)/2}{s + \epsilon} = \frac{s}{s + \epsilon}.$$

For the first row, the input mean square is $(9 + 16)10^{-8}/2 = 1.25 \cdot 10^{-7}$. The value under the square root is $1.125 \cdot 10^{-6}$. Epsilon is eight times the input mean square, so it makes a substantial contribution to the denominator.

$$\frac{1.25 \cdot 10^{-7}}{1.125 \cdot 10^{-6}} = \frac{1}{9} \approx 0.111111.$$

Now multiply the first row by ten to get $(3 \cdot 10^{-3}, 4 \cdot 10^{-3})$. Squaring the entries multiplies their mean square by 100. Epsilon stays fixed, giving

$$\frac{1.25 \cdot 10^{-5}}{1.25 \cdot 10^{-5} + 10^{-6}} = \frac{25}{27} \approx 0.925926.$$

The zero row has $s = 0$, and its denominator is $\sqrt{10^{-6}} = 10^{-3}$. Dividing each zero coordinate by this positive number leaves zero. In Python, compute one mean square per row by reducing the feature axis with `mean(dim=-1)`. Compare with the fractions above using float64 on CPU, an absolute tolerance of 10^{-12}, and zero relative tolerance.

```
from examples.ch08_modern.model import rms_norm

rms_input = torch.tensor(
    [[3e-4, 4e-4], [0.0, 0.0]], dtype=torch.float64
)
rms_scaled = rms_input.clone()
rms_scaled[0] *= 10
gains = torch.ones_like(rms_input[0])
epsilon = 1e-6
rms_atol = 1e-12

rms_cases = (
    ("original", rms_input, 1 / 9),
    ("first row x10", rms_scaled, 25 / 27),
)
for name, rows, expected in rms_cases:
    normalized = rms_norm(rows, gains, eps=epsilon)
    mean_squares = normalized.square().mean(dim=-1)
    torch.testing.assert_close(
        mean_squares, rows.new_tensor([expected, 0.0]),
        atol=rms_atol, rtol=0,
    )
    assert normalized.dtype == torch.float64
    assert torch.isfinite(normalized[1]).all()
    assert torch.count_nonzero(normalized[1]).item() == 0
    print(name)
    print(f"  input mean square (row 0)   {rows[0].square().mean():.9g}")
    print(f"  output mean square (row 0)  {mean_squares[0]:.12f}")
    print(f"  zero row output             {normalized[1].tolist()}")
```

```
OUTPUT
original
  input mean square (row 0)   1.25e-07
  output mean square (row 0)  0.111111111111
  zero row output             [0.0, 0.0]
first row x10
  input mean square (row 0)   1.25e-05
  output mean square (row 0)  0.925925925926
  zero row output             [0.0, 0.0]
```

The scaled row moves closer to mean square one because s now dominates ϵ. It is still below one because the added ϵ is positive. The output depends on the ratio ϵ/s for a nonzero row. The second row gives the same finite pair of zeros in both calls, since the scale calculation runs independently for each token.

7. Rebuild the worked gate

The chapter sends a width-four token through two separate projections. The gate branch should produce $g = (0, 2)$ and the feature branch should produce $u = (3, -1)$ on the input $x = (1, 0, 0, 0)$. With this input, each output coordinate of `nn.Linear` is just the weight attached to the first input coordinate. That lets us set both projections by hand.

PyTorch stores a linear layer's weight with one row per output coordinate and one column per input coordinate. It evaluates $xA^\top$ for a stored weight A. The gate and up weights are therefore each 2×4. Zero both matrices, then set the first column of the gate matrix to $(0, 2)^\top$ and the first column of the up matrix to $(3, -1)^\top$.

The chapter writes its down projection on the right of a two-coordinate row vector. Its mathematical matrix is

$$W_{down} = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & -1 \end{bmatrix}.$$

PyTorch needs four output rows and two input columns for `nn.Linear(2, 4)`. Copy $W_{down}^\top$ into the weight so that the linear layer computes the intended row-vector multiplication. Use `torch.no_grad()` while changing the weights in place. These assignments set the parameters and do not form part of a differentiable calculation.

```
from examples.ch08_modern.model import SwiGLU

gate_layer = SwiGLU(4, 2).double()
gate_input = torch.tensor(
    [1.0, 0.0, 0.0, 0.0], dtype=torch.float64
)
down_matrix = gate_input.new_tensor([
    [1.0, 0.0, 1.0, 0.0],
    [0.0, 1.0, 0.0, -1.0],
])
with torch.no_grad():
    gate_layer.gate_proj.weight.zero_()
    gate_layer.up_proj.weight.zero_()
    gate_layer.gate_proj.weight[:, 0] = gate_input.new_tensor([0, 2])
    gate_layer.up_proj.weight[:, 0] = gate_input.new_tensor([3, -1])
    gate_layer.down_proj.weight.copy_(down_matrix.T)

print("gate weight", gate_layer.gate_proj.weight.tolist())
print("up weight  ", gate_layer.up_proj.weight.tolist())
print("down weight", gate_layer.down_proj.weight.tolist())
```

OUTPUT

```
gate weight [[0.0, 0.0, 0.0, 0.0], [2.0, 0.0, 0.0, 0.0]]
up weight   [[3.0, 0.0, 0.0, 0.0], [-1.0, 0.0, 0.0, 0.0]]
down weight [[1.0, 0.0], [0.0, 1.0], [1.0, 0.0], [0.0, -1.0]]
```

The SiLU function multiplies its input by its sigmoid, $\text{SiLU}\,(a) = a/(1 + \exp(-a))$. Define $a_2 = 2/(1 + \exp(-2))$. The activated gate is $(0, a_2)$, with $a_2 \approx 1.761594155956$. Multiplying by the up features coordinate by coordinate gives $(0 \cdot 3, a_2 \cdot (-1)) = (0, -a_2)$.

The product's first coordinate weights the first row of W_{down} by zero. The second coordinate weights the second row by $-a_2$. The update is $(0, -a_2, 0, a_2)$. In particular, the positive fourth coordinate comes from multiplying the negative product by the matrix entry -1. The following check computes a_2 separately with the scalar formula and compares the intermediate tensors and full module output in CPU float64 with absolute tolerance 10^{-12} and zero relative tolerance.

```
from math import exp

gate_layer.eval()
with torch.inference_mode():
    gate_values = gate_layer.gate_proj(gate_input)
    up_values = gate_layer.up_proj(gate_input)
    gate_active = F.silu(gate_values)
    gate_product = gate_active * up_values
    gate_output = gate_layer(gate_input)

a2 = 2 / (1 + exp(-2))
gate_stages = (
    ("gate projection", gate_values, [0, 2]),
    ("up projection", up_values, [3, -1]),
    ("activated gate", gate_active, [0, a2]),
    ("elementwise product", gate_product, [0, -a2]),
    ("module output", gate_output, [0, -a2, 0, a2]),
)
for name, actual, expected in gate_stages:
    torch.testing.assert_close(
        actual, gate_input.new_tensor(expected), atol=1e-12, rtol=0
    )
    numbers = ", ".join(f"{value:.12g}" for value in actual)
    print(f"{name:<19} [{numbers}]")
```

OUTPUT

```
gate projection     [0, 2]
up projection       [3, -1]
activated gate      [0, 1.76159415596]
elementwise product [0, -1.76159415596]
module output       [0, -1.76159415596, 0, 1.76159415596]
```

8. Probe the attention mask

The two inputs begin with the same three tokens. The final token is `stopped` in the first sequence and `moved` in the second. Each output row holds 17 logits, meaning the decoder's scores for the 17 possible next tokens. The row at index 2 reads up through the shared third token and predicts a possible token at index 3. The observed token at index 3 is outside that row's allowed inputs.

Create a single model after setting seed 81, keep its weights fixed, and use evaluation mode. Run the inputs as integer IDs (`torch.long`) with a batch axis. The model's weights and logits use CPU float64. Compare the first three rows, then measure the difference at row 3. For the padding comparison, append `PAD=0` to the first sequence and compare just its four valid rows. The output shape changes from $[1, 4, 17]$ to $[1, 5, 17]$ when a position is appended.

Each reported maximum is the largest absolute change across the selected rows and all vocabulary coordinates. Apply `torch.testing.assert_close` to both invariance checks with `rtol=0` and `atol=1e-10`. In this check every logit can differ by at most 10^{-10}.

```
from examples.ch08_modern.model import ModernDecoder

with torch.random.fork_rng(devices=[]):
    torch.manual_seed(81)
    mask_model = ModernDecoder().double().eval()

base_ids = torch.tensor([[7, 8, 9, 10]], dtype=torch.long)
changed_ids = torch.tensor([[7, 8, 9, 16]], dtype=torch.long)
padded_ids = torch.tensor([[7, 8, 9, 10, 0]], dtype=torch.long)

with torch.inference_mode():
    base_logits = mask_model(base_ids)
    changed_logits = mask_model(changed_ids)
    padded_logits = mask_model(padded_ids)

mask_atol = 1e-10
torch.testing.assert_close(
    base_logits[:, :3], changed_logits[:, :3], rtol=0, atol=mask_atol
)
torch.testing.assert_close(
    base_logits, padded_logits[:, :4], rtol=0, atol=mask_atol
)

edit_delta = (changed_logits - base_logits).abs()
pad_delta = (padded_logits[:, :4] - base_logits).abs()
print("Base and padded shapes", base_logits.shape, padded_logits.shape)
print(f"Edit, first three rows   {edit_delta[:, :3].max():.12g}")
print(f"Edit, final row          {edit_delta[:, 3].max():.12g}")
print(f"Append PAD, valid rows   {pad_delta.max():.12g}")
```

```
OUTPUT
Base and padded shapes torch.Size([1, 4, 17]) torch.Size([1, 5, 17])
Edit, first three rows   0
Edit, final row          1.69309613396
Append PAD, valid rows   0
```

In the saved run, changing the final token alters its own logit row. Its embedding changes at the start, and the causal mask permits the final query to use the key at its own position. A difference is possible there. For these weights the maximum change is about 1.6931. The first three rows agree because every block masks later keys. The first block starts with identical embeddings at those three positions and can only draw their updates from earlier or current positions. The same argument applies in the next block. RMSNorm, the feed-forward network, and the vocabulary projection process positions individually, so those operations preserve the agreement.

Appending PAD preserves the positions and contents of the four real tokens. To inspect the two mask rules, read the Boolean grid for the padded input. Rows are queries and columns are keys. An entry of 1 allows that query to read the key. The mask combines the lower triangle of causal permissions with the validity of each key. The PAD column is zero even in the final row, where the causal rule alone would allow the diagonal.

```
from examples.ch05_transformer.model import build_masks

valid_tokens = padded_ids != mask_model.pad_id
_, allowed_keys = build_masks(valid_tokens, valid_tokens)
print("Validity mask", valid_tokens[0].tolist())
print("Allowed keys, one row per query")
print(allowed_keys[0, 0].to(torch.int64))
assert not allowed_keys[..., -1].any()
```

```
OUTPUT
Validity mask [True, True, True, True, False]
Allowed keys, one row per query
tensor([[1, 0, 0, 0, 0],
        [1, 1, 0, 0, 0],
        [1, 1, 1, 0, 0],
        [1, 1, 1, 1, 0],
        [1, 1, 1, 1, 0]])
```

For the first four queries, the newly appended key is already later than the query and therefore excluded by causality. The invariance check shows that adding right-padding preserves these particular outputs. Inspecting the full mask also checks the separate rule that a padding key is invalid. The model can still produce a logit row at the padded query position. A padded label is set to `IGNORE` in the loss calculation, as the next exercise shows.

9. Check a full run

Run the companion module from the repository root using `python -m examples.ch08_modern.run_examples`. It prints a JSON record with component checks and a decoder example. The notebook invokes the same command with the current kernel's Python executable and displays the relevant fields from the actual run.

The decoder gets a batch of two examples. Shifting *the red car stopped* supplies three input and label pairs. Shifting *the blue moved* supplies two. The example pads the input of the second row to length three and adds an `IGNORE` entry, encoded as `-100`, to its label row. The shape of the scores is $[2, 3, 17]$. It records two sequences, three output positions per sequence, and 17 vocabulary scores at each position. There are six output rows in total and five graded labels.

The module uses float64 on CPU for its decoder and runs one SGD update on this fixed batch with learning rate 0.01. It checks that the gradient tensor exists for each parameter and that every entry in each gradient tensor is finite before taking the step.

```
import json
import subprocess

run_modern = subprocess.run(
    [sys.executable, "-m", "examples.ch08_modern.run_examples"],
    cwd=repo_root, check=True, capture_output=True, text=True,
)
decoder_report = json.loads(run_modern.stdout)["decoder"]
update_report = decoder_report["update"]

print("Input IDs             ", decoder_report["inputs"])
print("Label IDs             ", decoder_report["labels"])
print("Graded labels         ", decoder_report["scored_labels"])
print("Logits shape          ", decoder_report["logits_shape"])
print("Parameters per block  ", decoder_report["parameters_per_block"])
print("Parameters total      ", decoder_report["parameters_total"])
print("All gradients finite  ", update_report["finite_gradients"])
print(f"Mean loss before       {update_report['loss_before']:.9f}")
print(f"Mean loss after        {update_report['loss_after']:.9f}")

assert decoder_report["logits_shape"] == [2, 3, 17]
assert decoder_report["scored_labels"] == 5
assert update_report["finite_gradients"] is True
```

OUTPUT

```
Input IDs             [[7, 8, 9], [7, 15, 0]]
Label IDs             [[8, 9, 10], [15, 16, -100]]
Graded labels         5
Logits shape          [2, 3, 17]
Parameters per block  2592
Parameters total      5744
All gradients finite  True
Mean loss before      3.245773867
Mean loss after       3.197667914
```

The mean loss falls from about 3.245774 to 3.197668 on the batch used for the update. This example checks that the forward pass, the loss, and backpropagation work together on those five label positions. The check for finite gradients allows zero entries, which can arise, for instance, in embedding rows for words absent from the batch.

To derive the parameter count, let $d = 16$ be the residual width, $m = 32$ the intermediate SwiGLU width, and $V = 17$ the vocabulary size. The projections are bias-free, so each linear layer contributes exactly its input width times its output width.

Part of one block	Weight calculation	Parameters
Query, key, value, and attention output projections	$4(16 \cdot 16)$	1,024
SwiGLU gate, up, and down projections	$3(16 \cdot 32)$	1,536
Attention and feed-forward RMSNorm gains	$2(16)$	32
One complete block	$1,024 + 1,536 + 32$	2,592

The attention projects all 16 coordinates before splitting them into two heads of width eight. That reshaping does not add another factor of two to the four matrix counts. SwiGLU has two $16 \to 32$ projections and one $32 \to 16$ projection. Each RMSNorm has one learned gain per feature. The residual additions, rotary angles, and Boolean masks add no learned parameters.

Outside the blocks, the embedding table contains $17 \cdot 16 = 272$ weights. The untied vocabulary projection has its own $16 \cdot 17 = 272$ weights. The final RMSNorm adds 16. The fixed part of the count is $272 + 272 + 16 = 560$. It stays the same when we change only the number of blocks.

$$\begin{aligned} P(2) &= 560 + 2(2,592) = 5,744, \\ P(3) &= 560 + 3(2,592) = 8,336. \end{aligned}$$

The predicted increase is 2,592. Check it by counting the scalar entries of the three-block model's parameter tensors. These are exact integer checks, independent of the weights' floating-point dtype.

```
model_width, ff_width, vocabulary_size = 16, 32, 17
block_count = 4 * model_width**2 + 3 * model_width * ff_width
block_count += 2 * model_width
outside_blocks = 2 * vocabulary_size * model_width + model_width

with torch.random.fork_rng(devices=[]):
    three_block_model = ModernDecoder(n_layers=3)
actual_three = sum(p.numel() for p in three_block_model.parameters())
expected_three = outside_blocks + 3 * block_count
two_block_total = decoder_report["parameters_total"]

assert decoder_report["parameters_per_block"] == block_count
assert two_block_total == outside_blocks + 2 * block_count
assert actual_three == expected_three
assert actual_three - two_block_total == block_count

print(f"Two blocks, measured         {two_block_total:,}")
print(f"Three blocks, predicted      {expected_three:,}")
print(f"Three blocks, measured       {actual_three:,}")
print(f"Increase                     {actual_three - two_block_total:,}")
```

OUTPUT

```
Two blocks, measured         5,744
Three blocks, predicted      8,336
Three blocks, measured       8,336
Increase                     2,592
```

Within each block, **self-attention mixes positions**. For a query at one position, it forms a weighted sum of value vectors at the allowed positions, so information from other tokens can enter that position's update. The projections that make the queries, keys, and values act on each token separately. It is the attention-weighted sum over keys, `weights @ v`, that combines their information.

SwiGLU expands features within a position. Both the gate and up projections send one token from 16 coordinates to 32. SiLU acts on the gate, the two results multiply feature by feature, and the down projection returns that same token's update to width 16. The down projection lets the result be added to the residual stream at that position.

10. Catch a reset position (optional)

Imagine reading four tokens in two chunks. The first chunk holds tokens 0 and 1. The second holds tokens 2 and 3, although they occupy slots 0 and 1 inside that chunk's tensor. Rotary position embeddings, or RoPE, rotate the features of each query or key using its position in the sequence. When a cache stores a new rotated key, its angle must come from its sequence position. Giving the second chunk the positions 0 and 1 again would repeat the angles already used by the first chunk.

To isolate the effect of position, give all four keys the same unrotated content $(1, 0)$. The tensor has shape $[1, 1, 4, 2]$, meaning one sequence in the batch, one attention head, four tokens, and two features per token. The two features make one pair with frequency 1 radian per position because the first pair uses $base^0 = 1$. At position n, the rotated key is $(\cos n, \sin n)$. For instance, position 0 keeps the original pair and position 1 turns it to approximately $(0.5403, 0.8415)$.

First rotate the four keys together. Then rotate the first two and the last two separately, using the same positions as in the full sequence. Concatenating with `dim=-2` joins the token axis, which has length 2 in each slice, while preserving the last axis of two features per key. All calculations run in CPU `torch.float64`. The equality checks use absolute tolerance 10^{-12} and relative tolerance 0.

```
from examples.ch08_modern.model import rotate_pairs

rope_atol = 1e-12
rope_keys = torch.tensor([1.0, 0.0], dtype=torch.float64)
rope_keys = rope_keys.reshape(1, 1, 1, 2).repeat(1, 1, 4, 1)
rope_positions = torch.arange(4)

rope_full = rotate_pairs(rope_keys, rope_positions)
rope_first = rotate_pairs(
    rope_keys[:, :, :2, :], rope_positions[:2]
)
rope_second = rotate_pairs(
    rope_keys[:, :, 2:, :], rope_positions[2:]
)
rope_split = torch.cat((rope_first, rope_second), dim=-2)

torch.testing.assert_close(
    rope_full, rope_split, atol=rope_atol, rtol=0
)
print("Shape of both versions:", list(rope_full.shape))
print("Max absolute difference:",
      (rope_full - rope_split).abs().max().item())
for position, (x, y) in enumerate(rope_full[0, 0].tolist()):
    print(f"  Key at {position}: ({x:+.9f}, {y:+.9f})")
```

OUTPUT

```
Shape of both versions: [1, 1, 4, 2]
Max absolute difference: 0.0
  Key at 0: (+1.000000000, +0.000000000)
  Key at 1: (+0.540302306, +0.841470985)
  Key at 2: (-0.416146837, +0.909297427)
  Key at 3: (-0.989992497, +0.141120008)
```

Each key rotates independently of the others, so the two calculations agree when the position supplied for each key is the same. Now give the second slice the wrong positions $[0, 1]$. The four stored keys then have angles $[0, 1, 0, 1]$ radians even though their sequence positions are $[0, 1, 2, 3]$.

Compare the keys with the query $(1, 0)$ at position $m = 3$. With the same content in each vector, its unscaled dot product with the key rotated at position n is

$$\begin{aligned} \tilde{\mathbf{q}}_3^\top \tilde{\mathbf{k}}_n &= \cos 3 \cos n + \sin 3 \sin n \\ &= \cos (n - 3). \end{aligned}$$

The full run therefore gives $[\cos 3, \cos 2, \cos 1, 1]$. In particular, the fourth key shares the query's angle and the score is the dot product of a unit vector with itself. Resetting the second chunk replaces its angular distances from the query, 1 and 0 radians, with 3 and 2 radians. The scores become $[\cos 3, \cos 2, \cos 3, \cos 2]$.

The next cell uses matrix multiplication to compute one dot product per key. Transposing the key tensor's last two axes gives a feature-by-key matrix for each batch item and head. A query row times this matrix produces four unscaled scores in token order.

```
rope_reset_second = rotate_pairs(
    rope_keys[:, :, 2:, :], rope_positions[:2]
)
rope_reset = torch.cat((rope_first, rope_reset_second), dim=-2)
rope_query = rotate_pairs(
    rope_keys[:, :, :1, :], rope_positions[3:]
)

rope_full_scores = (rope_query @ rope_full.transpose(-2, -1))
rope_split_scores = (rope_query @ rope_split.transpose(-2, -1))
rope_reset_scores = (rope_query @ rope_reset.transpose(-2, -1))

expected_full = (rope_positions.to(torch.float64) - 3).cos()
expected_reset = torch.tensor(
    [-3.0, -2.0, -3.0, -2.0], dtype=torch.float64
).cos()
for actual, expected in (
    (rope_full_scores, expected_full),
    (rope_split_scores, expected_full),
    (rope_reset_scores, expected_reset),
):
    torch.testing.assert_close(
        actual[0, 0, 0], expected, atol=rope_atol, rtol=0
    )

for name, scores in (
    ("All four together", rope_full_scores),
    ("Split with sequence indices", rope_split_scores),
    ("Split with reset", rope_reset_scores),
):
    values = ", ".join(f"{v:+.7f}" for v in scores[0, 0, 0])
    print(f"{name:27s} [{values}]")
```

OUTPUT

```
All four together           [-0.9899925, -0.4161468, +0.5403023, +1.0000000]
Split with sequence indices [-0.9899925, -0.4161468, +0.5403023, +1.0000000]
Split with reset            [-0.9899925, -0.4161468, -0.9899925, -0.4161468]
```

The first two scores stay the same because those keys keep their original angles. In the correct run, the query is close in angle to key 2 and aligned with key 3. After the reset, key 2 receives key 0's score and key 3 receives key 1's score. We held the content fixed, so this change comes entirely from the wrong position indices.

A cache can use the starting sequence index of a chunk plus the index within that chunk. For this example, the starting indices are 0 and 2, and a single new query uses the index 3 for the fourth token. RoPE's common-shift property still holds if all query and key positions move by the same amount. Here the query and the first two keys stay at their original positions while only the last two keys move. This probe checks the position calculation for rotated keys and the resulting dot products.

11. Isolate a residual gradient (optional)

In Exercise 1, a zero update let the pre-norm output keep the input $(3, 4)$, while the post-norm output became $(-1, 1)$. This time ask how the same scalar loss reacts to a small change in either input coordinate. For each placement, define the loss L as the sum of the two output coordinates. The gradient $\nabla_x L$ has two entries, one for each component of the input x.

Reuse the same LayerNorm formula and affine settings in both branches. LayerNorm subtracts the mean across the two features and divides by the square root of the mean squared deviation plus ϵ. With unit gains, zero offsets, and $\epsilon = 0$, the normalized vector at $(3, 4)$ is $(-0.5, 0.5)/0.5 = (-1, 1)$. The variance is 0.25, so the denominator in this example is positive.

The zero update is a function that returns a vector of zeros for every input. It contributes zero change when its input changes. In pre-norm, the normalized vector is fed to the zero update and the original input passes along the residual path to the sum. In post-norm, the residual sum also equals the input, and the output then passes through LayerNorm. Use the two functions below for both arrangements. The `normalized_shape=(2,)` argument selects the two features as the axis to normalize.

```
def exercise_layer_norm(vector):
    return F.layer_norm(
        vector,
        normalized_shape=(2,),
        weight=torch.ones_like(vector),
        bias=torch.zeros_like(vector),
        eps=0.0,
    )


def zero_sublayer(vector):
    return torch.zeros_like(vector)
```

Create two independent input tensors with gradient recording enabled. Use the same two functions to build each output.

```
residual_pre_input = torch.tensor(
    [3.0, 4.0], dtype=torch.float64, requires_grad=True
)
residual_post_input = torch.tensor(
    [3.0, 4.0], dtype=torch.float64, requires_grad=True
)
residual_pre_output = residual_pre_input + zero_sublayer(
    exercise_layer_norm(residual_pre_input)
)
residual_post_output = exercise_layer_norm(
    residual_post_input + zero_sublayer(residual_post_input)
)
```

Each call to `backward()` writes to the `.grad` field of that arrangement's input. Compare those two fields with the analytic gradients.

```
residual_atol = 1e-12
for name, inputs, output, expected in (
    ("Pre-norm", residual_pre_input, residual_pre_output, [1., 1.]),
    ("Post-norm", residual_post_input, residual_post_output, [0., 0.]),
):
    loss = output.sum()
    loss.backward()
    torch.testing.assert_close(
        inputs.grad, torch.tensor(expected, dtype=inputs.dtype),
        atol=residual_atol, rtol=0,
    )
    print(f"{name} output:   {output.detach().tolist()}")
    print(f"{name} loss:     {loss.item():.1f}")
    print(f"{name} gradient: {inputs.grad.tolist()}")
```

OUTPUT

```
Pre-norm output:   [3.0, 4.0]
Pre-norm loss:     7.0
Pre-norm gradient: [1.0, 1.0]
Post-norm output:   [-1.0, 1.0]
Post-norm loss:     0.0
Post-norm gradient: [0.0, 0.0]
```

The pre-norm loss is $L = x_1 + x_2$. Increasing either input coordinate by a small amount increases the loss by that same amount. The gradient is $(1, 1)$. More generally, the addition sends the gradient back through the residual path unchanged and also back through the update branch. The zero sublayer removes the update branch's contribution in this exercise, so the gradient we see is exactly the one from the residual path.

In post-norm, both centered coordinates share the same denominator. Writing $\mu = (x_1 + x_2)/2$ and s for the standard deviation, the sum after LayerNorm is

$$L = \frac{x_1 - \mu}{s} + \frac{x_2 - \mu}{s} = \frac{x_1 + x_2 - 2\mu}{s} = 0.$$

This identity holds for inputs near $(3, 4)$, where s remains positive. Perturbing either coordinate changes the mean along with the coordinates, so the numerator still cancels. The gradient of this locally constant loss is $(0, 0)$. This cancellation depends on the summed output and the equal unit gains. Post-norm sends the residual contribution through LayerNorm, so its gradient depends on the loss and on how normalization transforms the incoming gradient. The Python comparison uses CPU `torch.float64` with absolute tolerance 10^{-12} and relative tolerance 0.